Full Stack Engineering Internship Assessment

Practical Project Challenge: "TaskFlow" Resilient MERN Kanban Platform

MERN STACK | FULL STACK CANDIDATE | PRACTICAL + DEFENSE

Role: SDE Intern (Full Stack)
Timeline: 48-Hour Take-Home Window
Focus: State, Concurrency & Defense
Evaluation: Code Review + Live Defense

> ⚠️ **Generative AI Policy: Transparent Usage Permitted**
>
> Candidates are **fully authorized and encouraged** to utilize modern AI tools (Claude, ChatGPT, GitHub Copilot, Cursor). High-velocity software engineers use AI to ship faster.
>
> **The Core Requirement:** You must possess complete mastery over every single line of code submitted. You will be asked to explain runtime mechanics, alternative implementations, and trade-offs. Generating code you cannot defend or live-modify results in immediate disqualification.

1. PROJECT MISSION: "TASKFLOW" KANBAN ENGINE

Your objective is to design, implement, and deploy a responsive, highly resilient **Kanban Board Application** using the MERN stack (MongoDB, Express, React, Node.js). Beyond simple CRUD interfaces, production applications require handling **state desynchronization, concurrent drag/drop events, optimistic UI rollbacks, and race conditions**. This assignment evaluates your ability to build a system that remains solid under load.

2. CORE FUNCTIONAL SPECIFICATIONS

A. Board & Task Management (CRUD)

- **Columns:** 3 core columns: `To Do`, `In Progress`, `Done`.
- **Task Schema:** Title, Description, Priority (`Low`, `Medium`, `High`), Column ID, and explicit position `orderIndex`.
- **Reordering:** Move cards across columns and reorder vertically within columns (via Drag-and-Drop or clean button controls).

B. State Management & Optimistic UI

- **Instant Client Updates:** Dragging or moving a card updates the client state immediately without waiting for API roundtrips.
- **Graceful Rollback:** If the backend API call fails (network drop or simulated 500 error), the UI must instantly revert the card to its previous position with an error toast.

C. Race Conditions & Concurrency Defense

- **Rapid Interaction Guard:** Prevent data corruption or duplicate cards when a user rapidly moves cards between columns in quick succession.
- **Atomic Persistence:** Ensure database updates in MongoDB use atomic operators (e.g., `$set`, `$position`, or transaction blocks) rather than naive array overwrites.

D. Theming & Polish (Light / Dark)

- **Theme Engine:** Full Light and Dark mode coverage with compliant color contrast ratios across all cards, badges, and modals.
- **Zero-Flicker Persistence:** User theme preference must be stored in `localStorage` and hydrate without initial visual flicker on reload.

> 💡 **Recommended Tech Stack**
>
> **Frontend:** React (Vite recommended), Tailwind CSS, State Library (Zustand, Redux Toolkit, or Context + useReducer).
> **Backend:** Node.js, Express.js (REST or WebSocket), MongoDB (Mongoose or native driver). TypeScript is welcomed.

3. WILDCARD FEATURE: CANDIDATE'S CHOICE (PICK EXACTLY ONE)

Great engineers don't just follow specs; they think about product utility and user experience. Implement **one** of the following three features to distinguish your submission:

Option 1: Real-Time Debounced Multi-Filter & Search

A header search input debounced at 300ms that searches card titles/descriptions, paired with multi-select filter chips for Priority and Column. The filtered state must preserve card drag/move functionality seamlessly.

Option 2: Audit & Activity Log Trail

A slide-over drawer showing chronological history of actions: *"Task 'Setup Auth' moved to 'In Progress' by User at 14:22"*. Persisted in a MongoDB activity collection.

Option 3: Nested Checklist & Progress

Add subtask checklist items to any card. Include an animated progress bar on the card showing completion percentage (e.g., 3/5 subtasks completed = 60%).

> 💡 **Note on Additional Features**
>
> You are welcome to add extra features or enhancements beyond the core scope. However, please be aware that **any extra feature or library you include is subject to technical assessment**—you must thoroughly understand and be prepared to defend its architecture, dependencies, and code choices.

4. MANDATORY WRITTEN DELIVERABLE: `DECISIONS.MD`

In the root of your code repository, you must commit a markdown document named `DECISIONS.md`. This document reveals how you reason about software design, trade-offs, and failure modes. Answer each of the following four questions thoroughly:

1. State Architecture & Library Selection

Which state management approach did you choose (Zustand, Redux Toolkit, React Context, etc.)? Compare it against at least one alternative and explain why your choice was optimal for this problem. What are its architectural limitations?

2. Race Condition & Concurrency Defense Mechanism

Walk through how your codebase protects against race conditions. What happens if a user drags 5 cards within 1 second on a high-latency (3G) network? How do your frontend and backend guarantee that card ordering and column assignment remain consistent?

3. AI Debugging & Hallucination Retrospective

Provide at least one real example during this project where an AI tool (Claude, ChatGPT, Copilot) suggested inaccurate, suboptimal, or broken code. How did you spot the error, what was the underlying bug, and how did you resolve it?

4. High-Scale Failure Analysis

Imagine this board scales to 50,000 active tasks and 100 concurrent team members making edits simultaneously. What is the very first technical bottleneck that will break in your current code (DB query, event loop, re-render cascade)? How would you re-architect it?

5. EVALUATION RUBRIC (100 POINTS TOTAL)

Submissions will be scored objectively across five dimensions:

Evaluation Dimension	Assessment Criteria	Weight
Fullstack Architecture & Code Quality	Clean folder separation (client/server), idiomatic React patterns, proper REST API status codes, reusable utilities, clean MongoDB schemas.	**25 Pts**
Optimistic UI & Resilient State	Zero UI stutter during card moves, instantaneous visual feedback, reliable state rollback upon simulated network drops.	**25 Pts**
Concurrency & Theming Quality	Robust handling of rapid interactions, atomic MongoDB writes, persistent zero-flicker Dark/Light theme toggle.	**20 Pts**
Engineering Depth in `DECISIONS.md`	Clarity of technical explanations, honest discussion of AI tool usage, and thoughtful architectural trade-off evaluations.	**15 Pts**
Wildcard Feature Execution	Complete, polished, and bug-free implementation of candidate-selected feature demonstrating strong UX polish.	**15 Pts**

🎯 6. Live Technical Walkthrough & Code Defense (The Final Gate)

Candidates meeting the submission quality threshold will participate in a live technical review with engineering leads:

- **Deep Code Walkthrough:** We will point to an arbitrary hook, middleware, or controller: *"Walk us through the exact call stack and lifecycle of this logic from client render to database write."*
- **Live Code Modification:** A practical live tweak (e.g., adding a new priority badge and dynamic filter) to confirm genuine code ownership and reasoning.

7. MANDATORY GIT WORKFLOW & VERIFICATION REQUIREMENTS

- ✔ **Verification from `main` Branch Only: All code, tests, and documentation will be evaluated and verified strictly from the `main` branch.** Any work left unmerged in separate feature branches will not be evaluated.
- ✔ **Mandatory Data Seeder Script (`npm run seed`):** Your backend **must include an executable database seed script** (e.g., `npm run seed`). When executed, it must reset and populate MongoDB with realistic sample cards distributed across all 3 columns with varied priorities and tags so evaluators can immediately test your app with one command.
- ✔ **Public GitHub Repository with Feature Branching:** Must be hosted on a public GitHub repository. Development **must strictly follow a feature-branching workflow** (e.g., `feature/kanban-crud` , `feature/optimistic-ui` , `feature/theme-support`) merged into `main` branch via Pull Requests. Direct unbranched commits or a single bulk upload commit will lead to disqualification.
- ✔ **Conventional & Meaningful Commits:** Commit messages must be structured, descriptive, and atomic (e.g., `feat: implement optimistic rollback on card move failure` , `fix: race condition in orderIndex recomputation`).
- ✔ **Clear `README.md` & Deliverables:** Include environment templates (`.env.example`), clear run commands, and the mandatory `DECISIONS.md` file in the repository root. Submission within **48 hours**.